

Tutorial — LangChain, LangGraph y LangSmith

MMIA 6013 — IA Generativa y Agentes · Sesión 14, referencia rápida · jueves 1 de octubre de 2026

Instalación, clases y código ejecutado contra la H200 · para el Taller 3 y el Taller 2

Qué hace cada pieza, cuándo no hace falta, y cuánto cuesta

Daniel Andrés Riofrío Almeida

Tres piezas, tres trabajos, y ninguna es obligatoria

Pieza	Qué hace	Qué es
LangChain	habla con modelos, herramientas y recuperadores con una sola interfaz; trae <code>create_agent</code>	biblioteca, MIT
LangGraph	orquesta el bucle como grafo de estados: aristas, pausas, <i>checkpoints</i>	biblioteca, MIT
LangSmith	registra las trazas, guarda <i>datasets</i> y evalúa	un servicio en la nube; su cliente es MIT

- Los agentes de LangChain están contruidos sobre LangGraph: son el mismo bucle, más empaquetado
- El Taller 3 se aprueba con el bucle simple del lunes. Estas tres son extensión, no requisito

Fuente: docs.langchain.com/oss/python/langchain/overview («LangChain's agents are built on top of LangGraph») y docs.langchain.com/langsmith/observability-quickstart (consultados 2026-10-01); licencias leídas de los metadatos de los paquetes instalados con `proceso/scripts/verificar_tutorial_s14.py`; `curso/talleres/taller-03-agentes.md`, Parte 4

Instalación: lo que el Lab 03 ya fija y lo que se añade

- Lo del laboratorio trae `langgraph`, `langchain-openai` y `langchain-anthropic` con versión fija
- `create_agent` vive en el paquete `langchain`, y el cliente de trazas en `langsmith`: se añaden
- Todo se instaló sin conflictos sobre la versión del curso (`pip check`, 2026-10-01)

```
python -m venv .venv && source .venv/bin/activate
pip install -r curso/labs/Lab-03-Agentes/requirements.txt
# langgraph==1.2.12 langchain-openai==1.6.7 langchain-anthropic==1.7.5

pip install langchain==1.4.3 langsmith==0.14.2    # solo si los usan
pip check                                           # sin conflictos

python -c "import langgraph, langchain, langsmith; print('ok')"
```

LangChain: el modelo como un objeto, contra la H200

- **ChatOpenAI** habla con cualquier servidor compatible con OpenAI: la H200 lo es
- El id del modelo se pregunta al endpoint: no se escribe a mano
- **usage_metadata** trae los tokens de cada llamada: es lo que la traza tiene que guardar

```
import json, urllib.request
from langchain_openai import ChatOpenAI

URL = "http://172.28.230.10:12555/v1"          # GlobalProtect
with urllib.request.urlopen(f"{URL}/models") as r:
    modelo = json.load(r)["data"][0]["id"]    # no se escribe

llm = ChatOpenAI(base_url=URL, api_key="local", model=modelo,
                  temperature=0)
r = llm.invoke("¿Cuál es la capital de Ecuador? Una palabra.")
r.content      # 'Quito'
r.usage_metadata # {'input_tokens': 24, 'output_tokens': 34, ...}
```

@tool: la docstring es la descripción, y la firma es el esquema

- El decorador lee el nombre de la función, su docstring y sus tipos: es el contrato del lunes
- `bind_tools` le pasa el catálogo al modelo; el modelo **pide**, no ejecuta
- Es la corrección 1 del Taller 3, y la conexión de solo lectura es la 2

```
import sqlite3
from langchain_core.tools import tool

@tool
def consultar_sql(consulta: str) -> list:
    """Ejecuta UNA consulta SELECT de solo lectura sobre la base
    de ventas. Hasta 20 filas."""
    uri = "file:data/database.sqlite?mode=ro"
    with sqlite3.connect(uri, uri=True) as c:
        q = f"SELECT * FROM ({consulta}) LIMIT 20"
        return c.execute(q).fetchall()

consultar_sql.args # {'consulta': {'title': 'Consulta', 'type': 'string'}}
ai = llm.bind_tools([consultar_sql]).invoke("¿Cuántas ventas hay?")
ai.tool_calls      # [{'name': 'consultar_sql', 'args': {...}}]
```

Salida estructurada: el esquema solo sirve si el modelo lo lee

- `with_structured_output` devuelve un objeto validado: es el nodo clasificar del miércoles
- En la H200, el método por defecto no le muestra el esquema al modelo: 25 tokens con o sin él
- Con `method="function_calling"` el esquema viaja como herramienta: 245 tokens, y acierta

```
from typing import Literal
from pydantic import BaseModel, Field

class Ruta(BaseModel):
    """A qué nodo del grafo va la pregunta de un empleado."""
    ruta: Literal["sql", "rag", "sensible"] = Field(description=
        "sql: cifras de ventas · rag: políticas internas · "
        "sensible: borrar, modificar o enviar datos")
    motivo: str

clasificar = llm.with_structured_output(
    Ruta, method="function_calling")
clasificar.invoke("Borra los registros duplicados del cliente X").ruta
# 'sensible' · con el método por defecto: 'sql'
```

Fuente: `proceso/scripts/verificar_tutorial_s14.py`, p. 6, ejecutado el 2026-10-01 contra la H200: tokens de entrada 25 sin esquema, 25 con `method="json_schema"` y 245 con `method="function_calling"`; las tres preguntas de prueba se clasificaron bien solo con el segundo. `curso/actividades/_fuentes/s3-mie.py`, Parte 2 (nodo_clasificar)

Del Taller 2: el recuperador de bge-m3, como una herramienta más

- `OpenAIEmbeddings` habla con el Ollama de la H200: el mismo `bge-m3` del Taller 2
- `InMemoryVectorStore` basta para probar; su Qdrant del Taller 2 tiene su propio adaptador
- Envuelto en `@tool`, el RAG es una herramienta del agente: es el caso de investigación

```
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore

emb = OpenAIEmbeddings(base_url="http://172.28.230.10:11434/v1",
                        api_key="local", model="bge-m3:latest",
                        check_embedding_ctx_length=False)
vs = InMemoryVectorStore.from_texts(FRAGMENTOS, emb) # 1024 dim.

@tool
def buscar_documentos(pregunta: str) -> str:
    """Busca en las políticas internas el fragmento que responde."""
    return "\n".join(d.page_content
                     for d in vs.similarity_search(pregunta, k=1))
```

LangGraph: el bucle, escrito como grafo

14

MessagesState, ToolNode y tools_condition: el agente del Taller 3

- **MessagesState** es un estado con la lista de mensajes y su reductor ya puesto
- **ToolNode** ejecuta lo que el modelo pidió; **tools_condition** es la arista: herramienta o fin
- **recursion_limit** es el tope de pasos: al pasarlo, **GraphRecursionError**

```
from langgraph.graph import StateGraph, MessagesState, START
from langgraph.prebuilt import ToolNode, tools_condition

herramientas = [describir_esquema, consultar_sql]
llm_h = llm.bind_tools(herramientas)

def agente(estado: MessagesState):
    return {"messages": [llm_h.invoke(estado["messages"])]}

g = StateGraph(MessagesState)
g.add_node("agente", agente)
g.add_node("tools", ToolNode(herramientas))
g.add_edge(START, "agente")
g.add_conditional_edges("agente", tools_condition) # tools o END
g.add_edge("tools", "agente")
app = g.compile()
app.invoke({"messages": [("user", "¿Importe total de marzo de 2026?")]}),
          {"recursion_limit": 10})                # → 845.520,02
```

Fuente: `proceso/scripts/verificar_tutorial_s14.py`, p. 8, ejecutado el 2026-10-01: la respuesta contiene 845 520,02, que es la `sql_verificacion` de P1 en `curso/labs/Lab-03-Agents/preguntas_plantilla.json`, comprobada con el `comparador de evaluar.py`; 983 tokens de entrada en la corrida; `slides_v2/sesion-14/sesion-14.qmd`, pp. 9–14

create_agent: el mismo bucle en una línea, y lo que esconde

- `create_agent` arma ese mismo grafo: modelo, herramientas, prompt del sistema y *middleware*
- Devuelve un grafo compilado de LangGraph: se invoca, se pausa y se traza igual
- Lo que esconde es justo lo que el Taller 3 califica: la arista, el tope, el manejo de errores

```
from langchain.agents import create_agent

agente = create_agent(
    model=llm,
    tools=[describir_esquema, consultar_sql],
    system_prompt="Respondes con datos de la base. No inventes cifras.",
)
salida = agente.invoke({"messages": [
    {"role": "user", "content": "¿Importe total de marzo de 2026?"}]}))
salida["messages"][-1].content      # '... 845,520.02 ...'
type(agente).__name__                # 'CompiledStateGraph'
```

Cuándo usar cada uno

Si su agente...	Use	Porque
es un bucle con 3–6 herramientas	el bucle propio del lunes	es el baseline del taller y se ve entero
quiere el mismo bucle sin escribirlo	<code>create_agent</code>	es una línea, y sigue siendo LangGraph
bifurca, pausa para una persona o reanuda	LangGraph	la arista, el <code>interrupt</code> y el <code>checkpoint</code> son suyos
tiene que ser auditado o evaluado	LangSmith —o su propia traza—	lo que no queda escrito no se puede medir

— Lo que no compra ninguno: buenas descripciones de herramientas, ni permisos, ni el golden set

Fuente: docs.langchain.com/oss/python/langchain/overview (`create_agent` frente a LangGraph «for advanced needs combining deterministic and agentic workflows»), consultado 2026-10-01; [curso/actividades/semana-3/s3-mie-explicacion.html](#), 57–61 (regla del curso); [curso/talleres/taller-03-agentes.md](#), Parte 4, Opciones D y A

LangSmith: lo que la traza deja ver

14

LangSmith: trazas, datasets y evaluación, con dos variables de entorno

- Con LangChain o LangGraph, encender el rastreo es una variable: no se toca el código
- Para su propio bucle, `@traceable` en cada función, o `wrap_openai` sobre el cliente
- Además de trazas: *datasets* para el golden set y evaluaciones sobre ellos, que es la semana 4

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=...           # en .env, nunca en el repositorio
export LANGSMITH_PROJECT=taller-03     # opcional: agrupa las trazas
```

```
from langsmith import traceable

@traceable(name="consultar_ventas")
def consultar(mes: str) -> float:
    ...                               # cada llamada queda como un paso
```

Lo que cuesta: las bibliotecas, el modelo y el servicio

Qué	Cuánto	Con la H200
<code>langchain</code> , <code>langgraph</code> , <code>langsmith</code> (paquetes)	nada: licencia MIT	nada
el modelo	tokens de entrada y salida, al precio de su fila de la tabla semestral, con su fecha	cero
LangSmith, plan Developer	0 USD, 1 asiento, hasta 5 000 trazas base al mes	—
LangSmith, plan Plus	39 USD por asiento al mes, 10 000 trazas base incluidas, el resto por uso	—
retención de trazas	base, 14 días; extendida, 180, con recargo	—

- El precio por traza adicional no está en el texto publicado: se ve en el panel de facturación
- Una traza es una ejecución entera; y lleva sus prompts y sus datos a la nube de LangChain

Fuente: www.langchain.com/pricing y docs.langchain.com/langsmith/usage-and-billing, consultados 2026-10-01 (planes, trazas incluidas, retención; el precio por mil trazas adicionales no figura en el texto de ninguna de las dos); fuentes/modelos/modelos-2026-1.json (precios de modelo, con fecha por fila); licencias, proceso/scripts/verificar_tutorial_s14.py

Para el sábado: qué usar, y qué no hace falta

- El baseline se aprueba con el bucle simple: nada de esto es requisito
- Si usan LangChain: `@tool` con buenas docstrings, y `function_calling` en la salida estructurada
- Si usan LangGraph: la arista con su salida, `recursion_limit`, y el grafo dentro de su clase
- Si usan LangSmith: la clave en `.env`, y sus datos viajan fuera de la universidad
- Las versiones, fijadas en su `requirements.txt`: estos paquetes cambian cada semana
- Todo el código de este deck corre con `proceso/scripts/verificar_tutorial_s14.py`